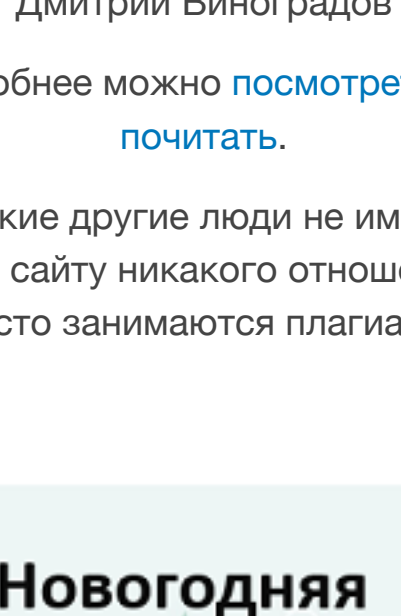


Language



Автор сайта



Дмитрий Виноградов

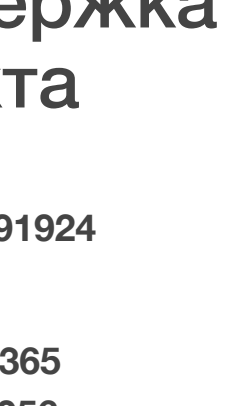
Подробнее можно [посмотреть](#) или [почитать](#).

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Новогодняя скидка 50% на набор курсов:

Jetpack Compose
Kotlin Coroutines
 Dagger + Hilt
Testing

Социальные сети



В канале я публикую ссылки на интересные и полезные статьи по Android

В [чате](#) можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте



Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551506702056

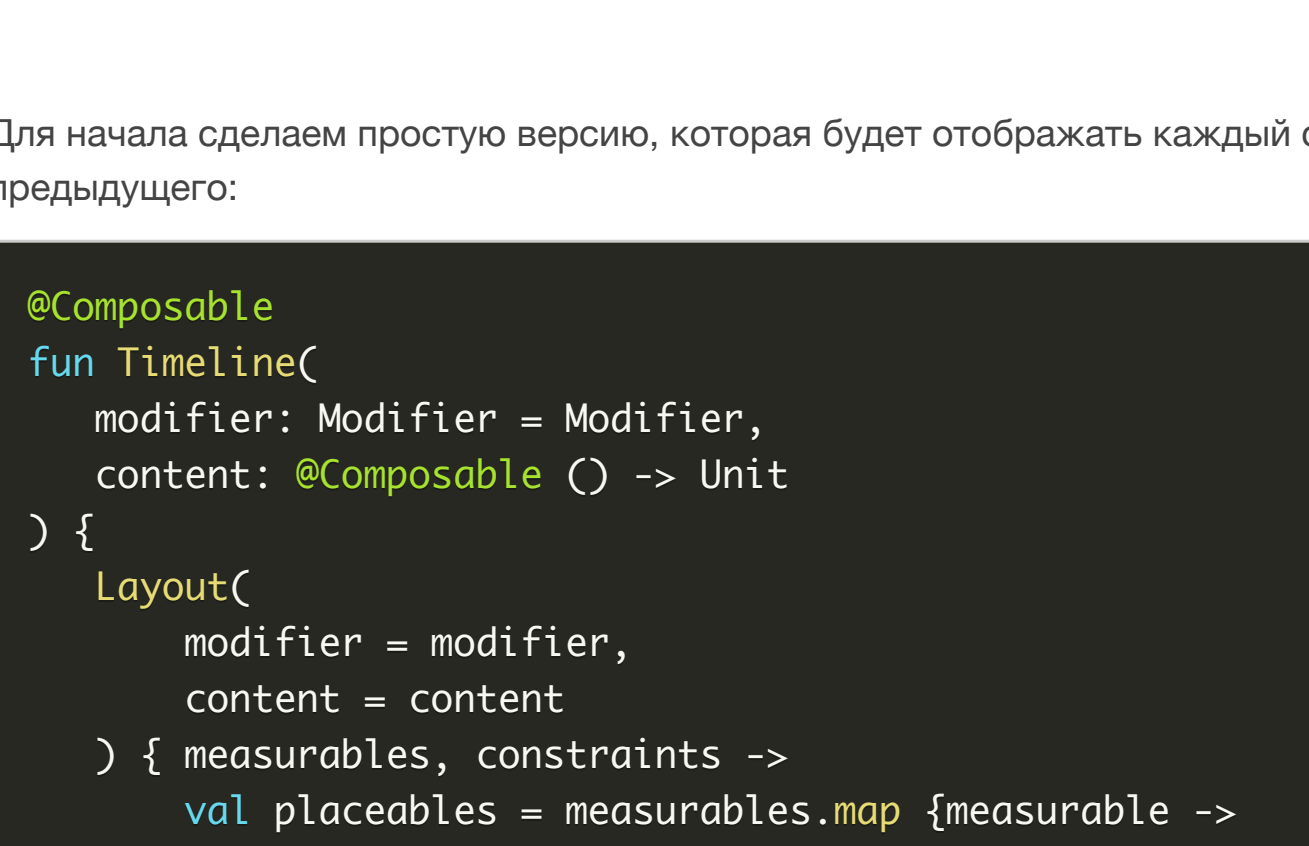
Paypal

Упок 26. Custom Layout - Timeline. Parent Data

08 июля 2023

В этом уроке создаем Custom Layout в виде Timeline и разбираемся, как родитель может получить данные от дочерних элементов с помощью Parent Data.

В продолжение темы Custom Layout давайте сделаем не аналог существующего Layout, а совершенно новый - Timeline Layout. Он может быть использован для отображения задач или процессов в таком виде:



Для начала сделаем простую версию, которая будет отображать каждый следующий дочерний элемент правее и ниже предыдущего:

```
@Composable
fun Timeline(
    modifier: Modifier = Modifier,
    content: @Composable () -> Unit
) {
    Layout(
        modifier = modifier,
        content = content
    ) { measurables, constraints ->
        val placeables = measurables.map { measurable ->
            measurable.measure(constraints) // Measure children
        }
        layout(constraints.maxWidth, constraints.maxHeight) { // Decide own size
            var x = 0
            var y = 0
            placeables.forEach { placeable ->
                placeable.placeRelative(x = x, y = y) // Place children
                x += placeable.width
                y += placeable.height
            }
        }
    }
}
```

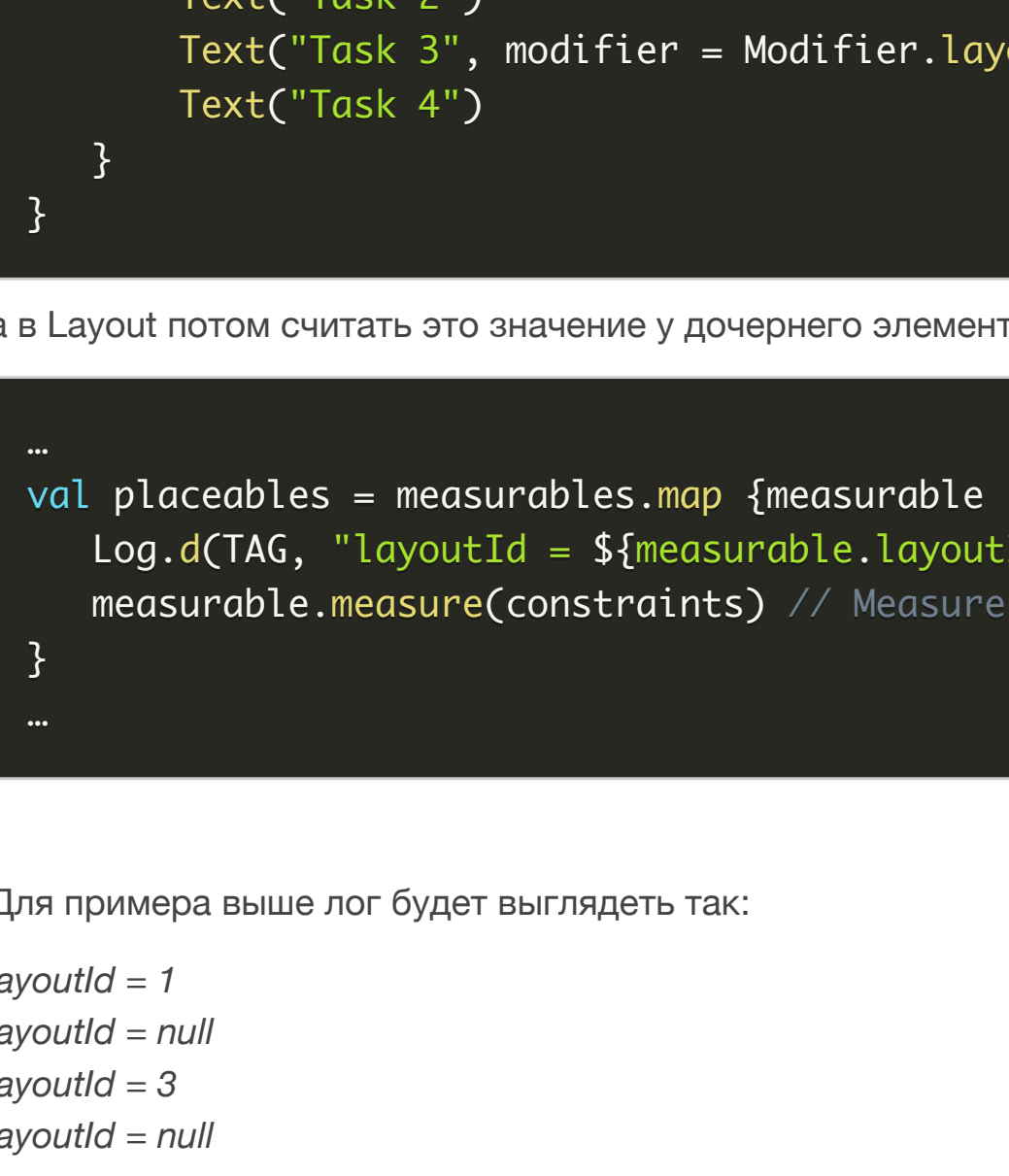
Основная работа здесь идет с placeable. Каждый размещаемый дочерний элемент сдвигает x на величину своей ширины и y на величину своей высоты. Т.е. ставит их в свой правый нижний угол. Следующий элемент будет размещен по этим x и y координатам, и он также сдвигает их в свой правый нижний угол. И так далее.

Этот алгоритм похож на Row и Column вместе взятые. Он размещает элементы друг за другом и по горизонтали и по вертикали.

Разместим несколько текстов в этом Layout:

```
@Composable
fun HomeScreen() {
    Timeline {
        Text("Task 1")
        Text("Task 2")
        Text("Task 3")
        Text("Task 4")
    }
}
```

Результат:



Работает, как и ожидалось. Каждый следующий элемент располагается правее и ниже предыдущего. Такая диаграмма отражает, что каждый следующий таск начинает выполняться после окончания предыдущего.

Теперь хотелось бы добавить пару фин в этот Layout.

Во-первых, нужна возможность указать, что определенный таск является последним в цепочке тасков. Это значит, что следующий таск начинает новую цепочку и должен быть расположен не правее предыдущего, а слева, в самом начале экрана. Назовем эту позицию - End.

Во-вторых, надо предусмотреть возможность того, что таски могут выполняться параллельно. Т.е. если текущий таск может работать параллельно, то следующий таск располагается не правее, а под текущим. Это будет позиция - Parallel.

Но чтобы это реализовать, надо чтобы Layout мог различать дочерние элементы между собой. Он должен знать, какой элемент располагать в обычной позиции, какой - в End, а какой - в Parallel.

Для этого дочерние элементы должны нести с собой данные о позиции, а Layout должен эти данные считывать. Самый простой вариант это сделать - использовать layoutId.

Layout ID

layoutId - Modifier оператор. Мы можем поместить в него какое-либо значение:

```
@Composable
fun HomeScreen() {
    Timeline {
        Text("Task 1", modifier = Modifier.layoutId("1"))
        Text("Task 2")
        Text("Task 3", modifier = Modifier.layoutId("3"))
        Text("Task 4")
    }
}
```

а в Layout потом считать это значение у дочернего элемента через measurable:

```
-
val placeables = measurables.map {measurable ->
    Log.d(TAG, "layoutId = ${measurable.layoutId}")
    measurable.measure(constraints) // Measure children
}
-
```

Для примера выше лог будет выглядеть так:

```
layoutId = 1
layoutId = null
layoutId = 3
layoutId = null
```

Таким образом мы присваиваем дочерним элементам данные, а Layout их считывает.

Используем этот механизм для передачи позиции таска в Layout. И давайте добавим еще таски в наш пример:

```
@Composable
fun HomeScreen() {
    Timeline {
        Text("Task 1")
        Text("Task 2", modifier = Modifier.layoutId("parallel"))
        Text("Task 3")
        Text("Task 4")
        Text("Task 5", modifier = Modifier.layoutId("end"))

        Text("Task 6")
        Text("Task 7", modifier = Modifier.layoutId("parallel"))
        Text("Task 8", modifier = Modifier.layoutId("parallel"))
        Text("Task 9")
        Text("Task 10")
    }
}
```

Мы хотим, чтобы Task 5 был последним в своей цепочке. Т.е. чтобы Task 6 начал новую цепочку. А таски Task 2, 7 и 8 могут выполняться параллельно с последующими.

Чтобы это сработало, нам надо считывать эти данные в нашем Layout и использовать их при расчете координат расположения дочерних элементов.

```
@Composable
fun Timeline(
    modifier: Modifier = Modifier,
    content: @Composable () -> Unit
) {
    Layout(
        modifier = modifier,
        content = content
    ) { measurables, constraints ->
        val placeables = measurables.map {measurable ->
            measurable.measure(constraints) // Measure children
        }
        layout(constraints.maxWidth, constraints.maxHeight) { // Decide own size
            var x = 0
            var y = 0
            placeables.forEach { placeable ->
                placeable.placeRelative(x = x, y = y) // Place children
                val layoutId = (placeable as? Measurable)?.layoutId
                when (layoutId) {
                    "parallel" -> {}
                    "end" -> x = 0
                    null -> x += placeable.width
                }
                y += placeable.height
            }
        }
    }
}
```

Смотрим, что происходит в цикле placeables.forEach

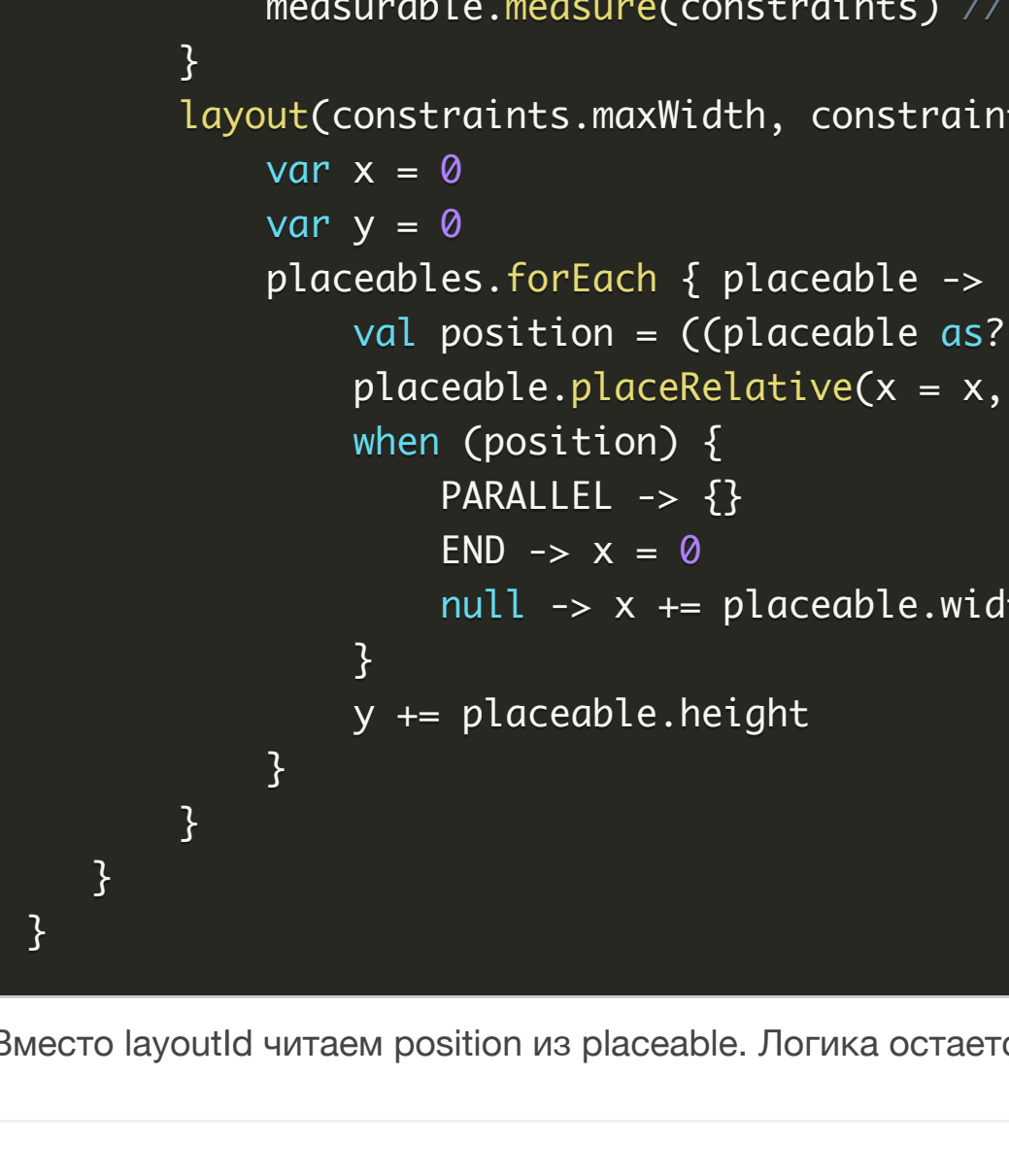
Т.х. данные о позиции влияют не на текущий элемент, а на последующий за ним, то текущий элемент мы спокойно разместим (placeable и measurable под капотом являются одним и тем же объектом. Поэтому приводим placeable к Measurable и читаем из него layoutId. Таким образом мы получаем позицию таска, которая влияет на то, где будет располагаться следующий таск.

Если позиция Parallel, то не меняем X. Следующий элемент будет расположен ровно под текущим, а не справа от него.

Если позиция End, значит следующий таск должен начать новую цепочку. Т.е. он должен располагаться слева, в самом начале экрана. Для этого сбрасываем X в 0.

Если позиция не указана, то действуем по обычной схеме - добавляем к X размеры текущего элемента. Следующий будет расположен правее.

Результат:



ParentData

Чтобы передавать в Layout данные о позиции его дочерних элементов (End или Parallel) мы использовали layoutId. Такой код выглядит не очень корректно. Имеет смысл создать свой кастомный вариант LayoutId, который будет заточен под наш тип данных.

Для начала создадим тип данных, который будем использовать, чтобы указать позицию:

```
enum class Position {
    END, PARALLEL
}
```

Теперь создадим специальный ParentDataModifier под наш тип данных:

```
class PositionParentData(
    val position: Position
): ParentDataModifier {
    override fun modifyParentData(parentData: Any?): Any? =
        this as? PositionParentData
}
```

Он позволит нам создать свой Modifier оператор, чтобы передавать туда данные о позиции:

```
fun Modifier.position(position: Position) = this.then(PositionParentData(position))
```

Заменяем layoutId на свой оператор position:

```
@Composable
fun HomeScreen() {
    Timeline {
        Text("Task 1")
        Text("Task 2", modifier = Modifier.position(PARALLEL))
        Text("Task 3")
        Text("Task 4")
        Text("Task 5", modifier = Modifier.position(END))

        Text("Task 6")
        Text("Task 7", modifier = Modifier.position(PARALLEL))
        Text("Task 8", modifier = Modifier.position(PARALLEL))
        Text("Task 9")
        Text("Task 10")
    }
}
```

И подправляем код в Layout:

```
@Composable
fun Timeline(
    modifier: Modifier = Modifier,
    content: @Composable () -> Unit
) {
    Layout(
        modifier = modifier,
        content = content
    ) { measurables, constraints ->
        val placeables = measurables.map {measurable ->
            measurable.measure(constraints) // Measure children
        }
        layout(constraints.maxWidth, constraints.maxHeight) { // Decide own size
            var x = 0
            var y = 0
            placeables.forEach { placeable ->
                val position = ((placeable as? Measurable)?.parentData as? PositionParentData)?.position
                placeable.placeRelative(x = x, y = y) // Place children
                when (position) {
                    PARALLEL -> {}
                    END -> x = 0
                    null -> x += placeable.width
                }
                y += placeable.height
            }
        }
    }
}
```

Вместо layoutId читаем position из placeable. Логика остается та же.

Присоединяйтесь к нам в [Telegram](#):

- в канале [StartAndroid](#) публикуются ссылки на новые статьи с сайта [startandroid.ru](#) и интересные материалы с хабра, medium.com и т.п.
- в чатах решаем возникающие вопросы и проблемы по различным темам: [Android](#), [Compose](#), [Kotlin](#), [RxJava](#), [Dagger](#), [Тестирование](#), [Performance](#)
- ну и если просто хочется поговорить с коллегами по разработке, то есть чат [Флудилня](#)